

# STAR ESP32 Firmware Implementation Primer

ESP32-S3 Motor Control and Sensor Integration Guide

STAR Development Team

December 2025

## Contents

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Project Overview</b>                        | <b>2</b> |
| 1.1      | Purpose . . . . .                              | 2        |
| 1.2      | Current Implementation Status . . . . .        | 2        |
| <b>2</b> | <b>Architecture Overview</b>                   | <b>2</b> |
| 2.1      | Dependency Inversion Principle (DIP) . . . . . | 2        |
| 2.2      | Directory Structure . . . . .                  | 2        |
| <b>3</b> | <b>Motor Control Architecture</b>              | <b>3</b> |
| 3.1      | Motor Control Refactoring Plan . . . . .       | 3        |
| 3.1.1    | lib/star_motor (Proposed) . . . . .            | 3        |
| 3.1.2    | lib/star_encoder (Proposed) . . . . .          | 4        |
| 3.1.3    | lib/star_pid (Proposed) . . . . .              | 4        |
| 3.2      | Core Communication Interfaces . . . . .        | 5        |
| 3.2.1    | SPI Peripheral Driver . . . . .                | 5        |
| <b>4</b> | <b>Testing Plan</b>                            | <b>5</b> |
| 4.1      | Unit Tests . . . . .                           | 5        |
| 4.2      | Integration Tests . . . . .                    | 6        |
| 4.3      | Acceptance Criteria . . . . .                  | 6        |
| <b>5</b> | <b>Build and Flash Instructions</b>            | <b>6</b> |
| 5.1      | Prerequisites . . . . .                        | 6        |
| 5.2      | Build Commands . . . . .                       | 6        |
| 5.3      | Configuration . . . . .                        | 6        |
| <b>6</b> | <b>References</b>                              | <b>7</b> |

# 1 Project Overview

## 1.1 Purpose

The `star-esp32-firmware` project provides real-time motor control, sensor integration, and communication with the Raspberry Pi 5 gateway for the STAR robot platform. It runs on the ESP32-S3 microcontroller using FreeRTOS.

## 1.2 Current Implementation Status

| Component                 | Status   | Notes                                  |
|---------------------------|----------|----------------------------------------|
| STAR Framework Core       | Complete | DIP architecture, interfaces           |
| Bus Abstraction Layer     | Complete | I2C, SPI, 1-Wire, SMBus (slimmed down) |
| Essential Drivers         | Complete | BQ7850 (BMS), DS18B20 (Temp)           |
| Error Handling            | Complete | Retry logic with exponential backoff   |
| Pin Validation            | Complete | GPIO conflict detection                |
| Motor Control (MCPWM)     | Complete | Center-aligned PWM for 4 motors        |
| SPI Peripheral (Pi5 comm) | Complete | Binary protocol for commands           |
| PID Controller            | Complete | Integrated into motor control loop     |
| Encoder Reading           | Complete | Integrated into motor control loop     |

Table 1: Implementation Status Summary

# 2 Architecture Overview

## 2.1 Dependency Inversion Principle (DIP)

The firmware follows SOLID principles with emphasis on Dependency Inversion:

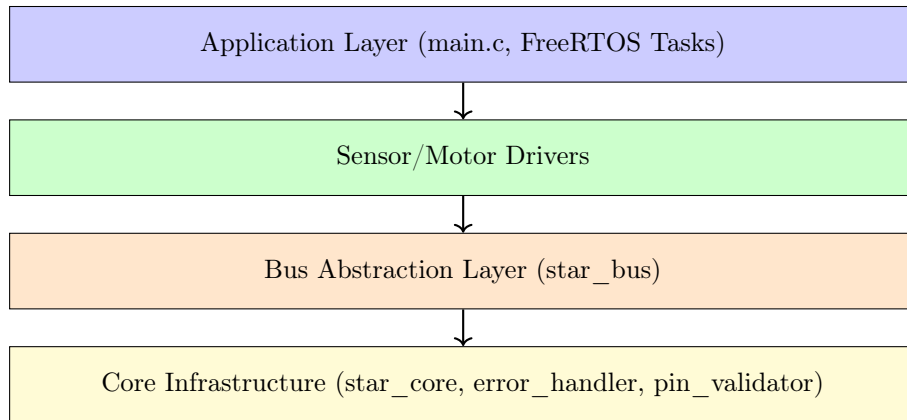


Figure 1: STAR Firmware Layered Architecture

## 2.2 Directory Structure

```

1 star-esp32-firmware/
2 |-- src/                                # Application entry point (currently empty)

```

```

3 |
4 |-- lib/
5 |   |-- star_core/           # Abstract interfaces
6 |   |-- star_error_handler/  # Error handling
7 |   |-- star_pin_validator/  # GPIO conflict detection
8 |   |-- star_bus/           # Unified bus abstraction
9 |   |-- star_motor/         # Motor, Encoder, and PID control
10 |   +-- star_bms_bq7850/     # Battery management
11 |
12 |-- platformio.ini          # Build configuration
13 +-- sdkconfig.*              # ESP-IDF configs

```

## 3 Motor Control Architecture

### 3.1 Motor Control Refactoring Plan

The `star_motor` library currently handles all motor-related functionality. For improved modularity, testability, and clarity, it is recommended to refactor this monolithic library into three distinct components as originally envisioned. The following sections outline the proposed architecture for each module.

#### 3.1.1 lib/star\_motor (Proposed)

This library should be responsible only for low-level motor control using the ESP32's MCPWM peripheral. Its sole focus is to abstract the hardware details of generating PWM signals.

1. Create a clean interface with motor handle and configuration structs.
2. Implement MCPWM initialization for 4 motors (2 MCPWM units, 2 operators each).
3. Configure center-aligned (up-down count) mode for reduced EMI.
4. Implement dead-time insertion to prevent H-bridge shoot-through.
5. Add fault input handling for overcurrent protection.

#### Key Parameters:

- PWM Frequency: 20 kHz (above audible range)
- Dead Time: 500ns (DRV8243 requirement)
- Duty Cycle Resolution: 0.1%

Listing 1: Proposed Motor Driver Interface

```

1 typedef struct {
2     mcpwm_unit_t unit;
3     mcpwm_timer_t timer;
4     gpio_num_t pwm_a;
5     gpio_num_t pwm_b;
6     gpio_num_t fault_pin;
7     float max_duty;

```

```

8 } star_motor_config_t;
9
10 esp_err_t star_motor_init(star_motor_handle_t* handle,
11                           const star_motor_config_t* config,
12                           star_error_interface_t* error_iface);
13 esp_err_t star_motor_set_duty(star_motor_handle_t* handle, float
14                               duty_cycle);
14 esp_err_t star_motor_brake(star_motor_handle_t* handle);
15 esp_err_t star_motor_coast(star_motor_handle_t* handle);

```

### 3.1.2 lib/star\_encoder (Proposed)

This library would be dedicated to handling velocity feedback from the quadrature encoders.

1. Use the ESP32 PCNT (Pulse Counter) peripheral.
2. Configure for quadrature decoding (4x resolution).
3. Implement velocity calculation from the count delta over time.
4. Add overflow handling for the 16-bit hardware counter.

#### Encoder Specifications:

- Resolution: 341.2 counts/revolution
- Wheel diameter: 100mm
- Distance per count: 0.92mm

### 3.1.3 lib/star\_pid (Proposed)

This library should be a generic, reusable PID controller for closed-loop control. It should contain no hardware-specific code.

Listing 2: Proposed PID Controller Interface

```

1 typedef struct {
2     float kp, ki, kd;           % Gains
3     float integral_limit;       % Anti-windup
4     float output_limit;         % Max output
5     float deadband;             % Ignore small errors
6 } star_pid_config_t;
7
8 void star_pid_init(star_pid_t* pid, const star_pid_config_t* config);
9 float star_pid_update(star_pid_t* pid, float setpoint, float measurement,
10                       float dt);
11 void star_pid_reset(star_pid_t* pid);

```

#### Initial Tuning Values (adjust experimentally):

- $K_p = 1.0$ ,  $K_i = 0.5$ ,  $K_d = 0.1$
- Loop rate: 1000 Hz
- Integral limit:  $\pm 100$

## 3.2 Core Communication Interfaces

### 3.2.1 SPI Peripheral Driver

The firmware implements SPI peripheral mode for communication with the Raspberry Pi 5.

- **Mode:** Peripheral on SPI2 (HSPI)
- **Protocol:** Custom binary packet structure
- **Performance:** DMA is used for efficient, non-blocking transfers

SPI Pin Mapping:

| Signal | Pi5 GPIO | ESP32 GPIO |
|--------|----------|------------|
| COPI   | GPIO11   | GPIO13     |
| CIPO   | GPIO9    | GPIO12     |
| SCLK   | GPIO11   | GPIO14     |
| CS     | GPIO8    | GPIO15     |

Command Protocol:

Listing 3: SPI Command Protocol

```

1 // Command byte definitions
2 #define CMD_SET_VELOCITY      0x01 // Set wheel velocities
3 #define CMD_EMERGENCY_STOP   0x02 // Immediate stop
4 #define CMD_GET_ENCODERS     0x03 // Read encoder counts
5 #define CMD_GET_TELEMETRY    0x06 // Battery, temp, status
6
7 // Packet format: [CMD (1)] [LENGTH (1)] [DATA (N)] [CRC8 (1)]
8 typedef struct __attribute__((packed)) {
9     uint8_t command;
10    uint8_t length;
11    uint8_t data[32];
12    uint8_t crc8;
13 } spi_packet_t;
14
15 // Velocity command payload
16 typedef struct __attribute__((packed)) {
17     float left_velocity; // m/s
18     float right_velocity; // m/s
19     uint32_t timestamp; // ms since boot
20 } velocity_cmd_t;

```

## 4 Testing Plan

### 4.1 Unit Tests

1. **PID Controller:** Test with simulated plant, verify convergence
2. **Encoder Driver:** Test count accuracy, overflow handling

3. **Motor Driver:** Test PWM generation, dead-time
4. **SPI Protocol:** Test packet parsing, CRC validation

## 4.2 Integration Tests

1. **Motor + Encoder:** Closed-loop step response test
2. **SPI + Motor:** Command-to-motion latency measurement
3. **Full System:** Velocity tracking accuracy at various speeds

## 4.3 Acceptance Criteria

| Metric                  | Target               |
|-------------------------|----------------------|
| PID loop rate           | 1000 Hz $\pm$ 1%     |
| Velocity tracking error | < 5% at steady state |
| Command latency (SPI)   | < 1ms                |
| Emergency stop time     | < 10ms               |

# 5 Build and Flash Instructions

## 5.1 Prerequisites

```

1 # Install PlatformIO
2 pip install platformio
3
4 # Install ESP-IDF (handled by PlatformIO)
5 pio pkg install

```

## 5.2 Build Commands

```

1 # Build for ESP32-S3
2 pio run -e esp32s3
3
4 # Build for ESP32-WROOM (dev board)
5 pio run -e esp32_wroom
6
7 # Upload firmware
8 pio run -e esp32s3 --target upload
9
10 # Monitor serial output
11 pio device monitor -b 115200

```

## 5.3 Configuration

Edit `sdkconfig.esp32s3` for production settings:

- Enable brownout detection

- Set CPU frequency to 240 MHz
- Configure flash size (16 MB)
- Enable PSRAM if needed

## 6 References

- ESP-IDF MCPWM Documentation: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/mcpwm.html>
- ESP-IDF SPI Peripheral: [https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi\\_slave.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi_slave.html)
- FreeRTOS Task Notifications: <https://www.freertos.org/RTOS-task-notifications.html>
- DRV8243 Datasheet: TI SLVSFN4
- STAR FTP Section 16: Embedded Implementation
- STAR Software Architecture Primer: Section 12 (ESP32 Firmware)